

BÀI TOÁN THỎA MÃN RÀNG BUỘC (CONSTRAINT SATISFACTION PROBLEMS)

CHƯƠNG 5

Phác thảo

- ◇ Ví dụ về CSP
- ◇ Tìm kiếm quay lui cho CSP
- ◇ Cấu trúc vấn đề và phân rã vấn đề
- ◇ Tìm kiếm cục bộ cho CSP

Các vấn đề về sự thỏa mãn ràng buộc (CSP)

Vấn đề tìm kiếm tiêu chuẩn:

state là một “hộp đen”—bất kỳ cấu trúc dữ liệu cũ nào

hỗ trợ kiểm tra mục tiêu, đánh giá, kế thừa

CSP:

state được xác định bởi *variables* X_i với giá trị từ miền D_i

kiểm tra mục tiêu là một tập hợp các ràng buộc chỉ định

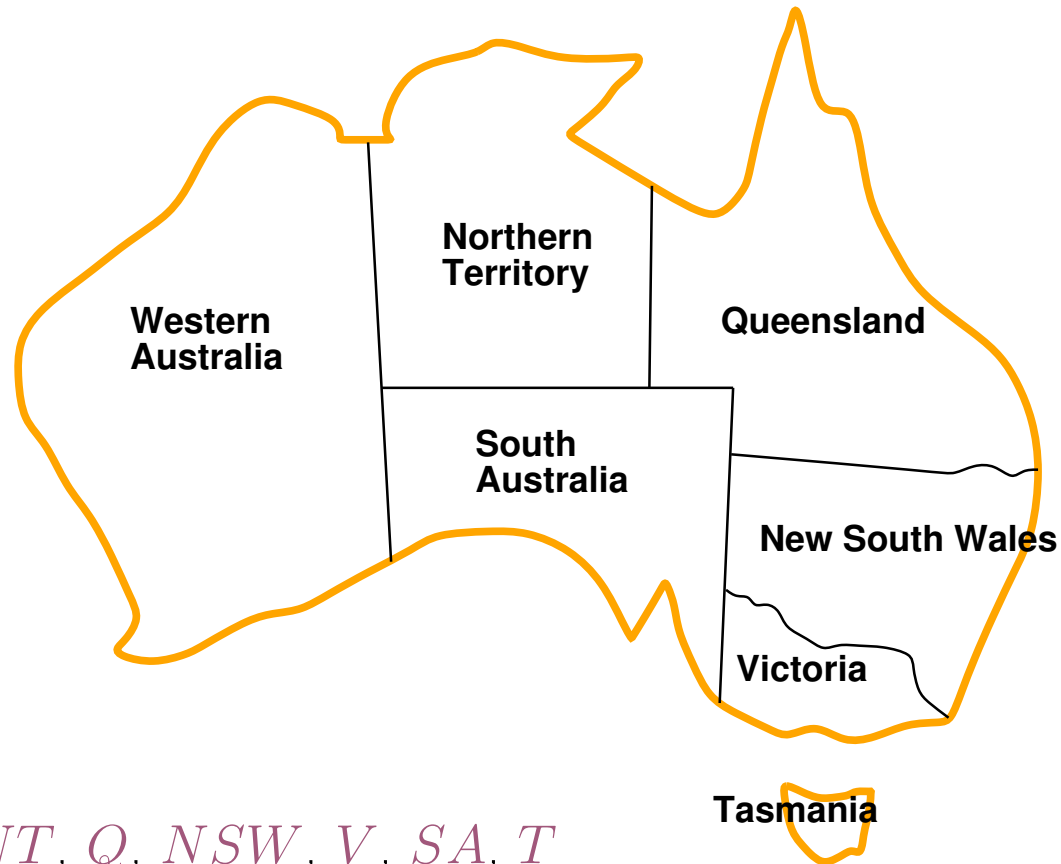
sự kết hợp các giá trị được phép cho các tập hợp con của biến

Ví dụ đơn giản về **ngôn ngữ biểu diễn hình thức**

Cho phép các thuật toán **có mục đích chung** hữu ích với nhiều sức mạnh hơn

hơn các thuật toán tìm kiếm tiêu chuẩn

Ví dụ: Tô màu bản đồ



Biến WA, NT, Q, NSW, V, SA, T

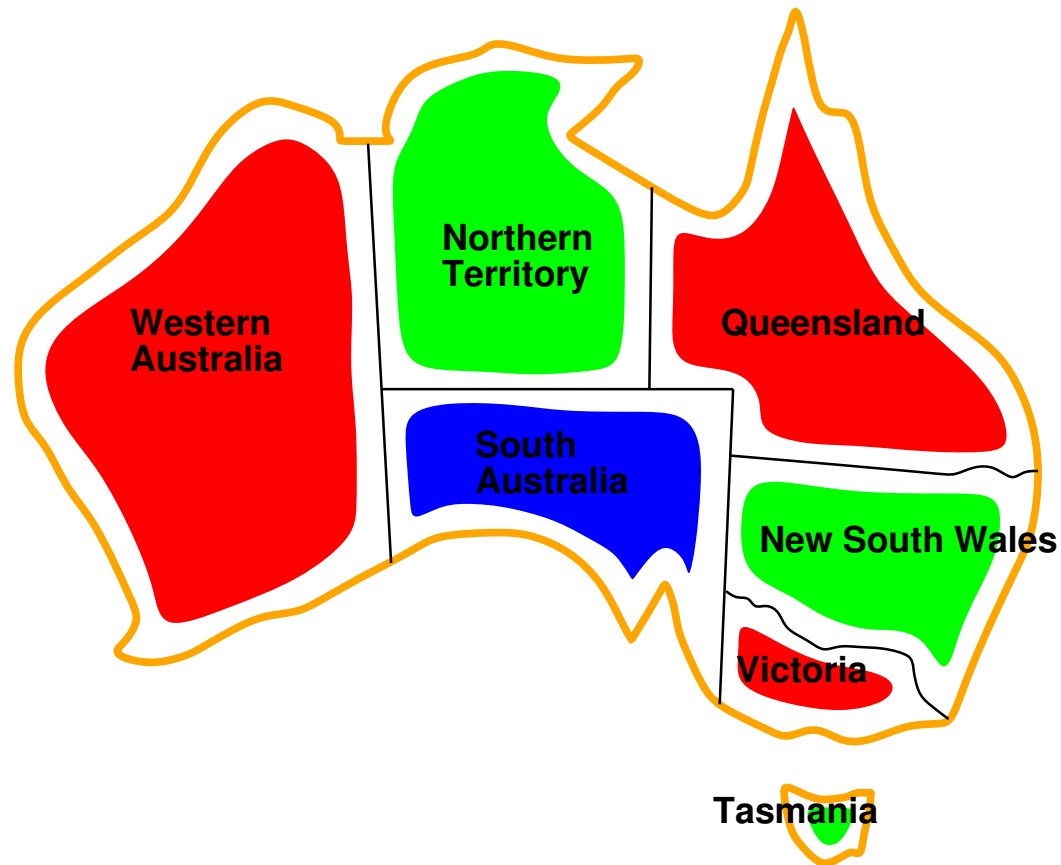
Miền $D_i = \{red, green, blue\}$

Ràng buộc: các vùng liền kề phải có màu khác nhau

ví dụ: $WA \neq NT$ (nếu ngôn ngữ cho phép điều này) hoặc

$(WA, NT) \in \{(red, green), (red, blue), (green, red), (green, blue), \dots\}$

Ví dụ: Tiếp theo tô màu bản đồ.



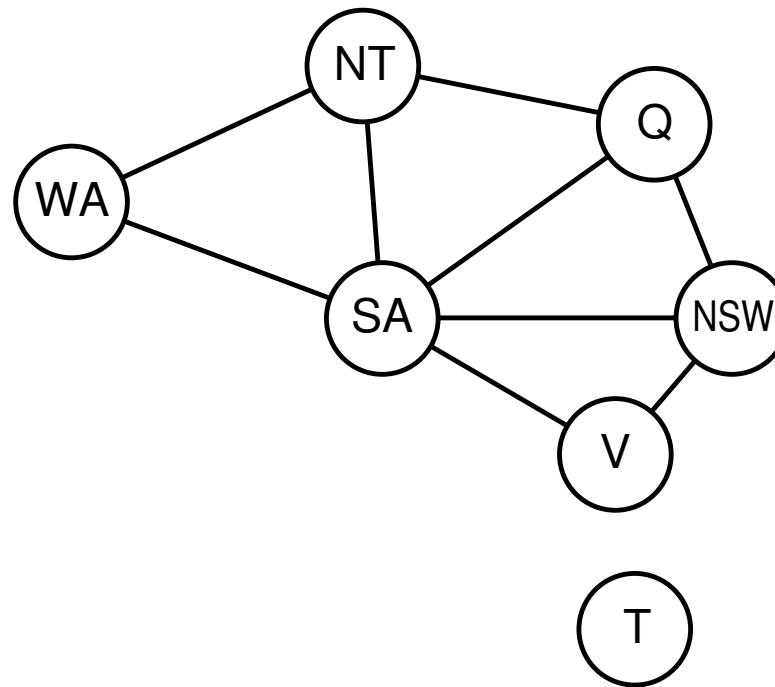
Giải pháp là các bài tập đáp ứng tất cả các ràng buộc, ví dụ:

$\{WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = green\}$

Biểu đồ ràng buộc

CSP nhị phân: mỗi ràng buộc liên quan đến nhiều nhất hai biến

Biểu đồ ràng buộc: các nút là các biến, các cung hiển thị các ràng buộc



Các thuật toán CSP có mục đích chung sử dụng cấu trúc biểu đồ

để tăng tốc độ tìm kiếm. Ví dụ: Tasmania là một bài toán con độc lập!

Các loại CSP

Biến rời rạc

miền hữu hạn; kích thước $d \Rightarrow O(d^n)$ hoàn thành bài tập

◇ ví dụ: CSP Boolean, bao gồm Khả năng thỏa mãn Boolean (NP-đầy đủ)

miền vô hạn (số nguyên, chuỗi, v.v.)

◇ ví dụ: lập kế hoạch công việc, các biến là ngày bắt đầu/kết thúc cho mỗi công việc

◇ cần ngôn ngữ ràng buộc, ví dụ: $StartJob_1 + 5 \leq StartJob_3$

◇ ràng buộc tuyến tính có thể giải quyết được, phi tuyến tính không thể giải quyết được

Biến liên tục

◇ ví dụ: thời gian bắt đầu/kết thúc cho các quan sát bằng Kính viễn vọng Hubble

◇ ràng buộc tuyến tính có thể giải được trong nhiều thời gian bằng phương pháp LP

Các loại ràng buộc

Các ràng buộc **Unary** liên quan đến một biến duy nhất,

ví dụ: $SA \neq green$

Các ràng buộc **Binary** liên quan đến các cặp biến,

ví dụ: $SA \neq WA$

Ràng buộc bậc cao hơn liên quan đến 3 biến trở lên,

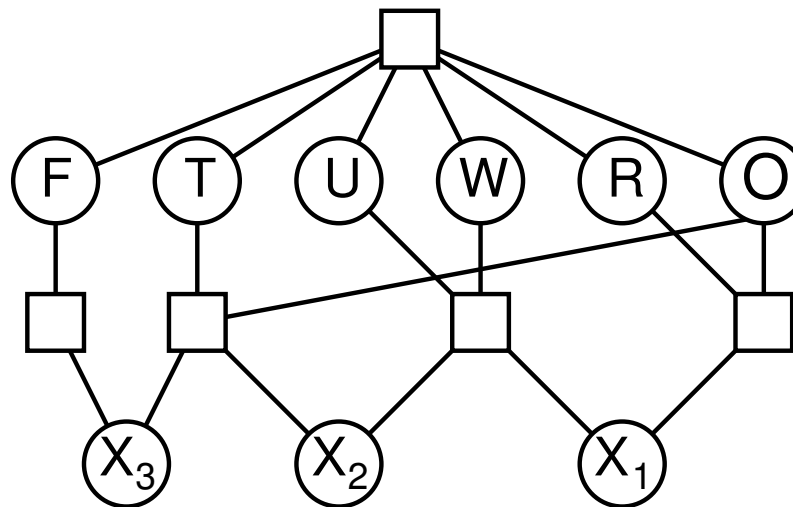
ví dụ: các ràng buộc cột mật mã

Preferences (ràng buộc mềm), ví dụ: red tốt hơn $green$
thường được biểu thị bằng chi phí cho mỗi lần gán biến

→ các vấn đề tối ưu hóa bị ràng buộc

Ví dụ: Số học mật mã

$$\begin{array}{r} \text{ T W O} \\ + \text{ T W O} \\ \hline \text{ F O U R} \end{array}$$



Biến: $F T U W R O X_1 X_2 X_3$

Miền: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Ràng buộc

$alldiff(F, T, U, W, R, O)$

$O + O = R + 10 \cdot X_1, \text{ v.v.}$

CSP trong thể giới thực

Vấn đề về bài tập

ví dụ: ai dạy lớp nào

Vấn đề về thời gian biểu

ví dụ: lớp nào được cung cấp khi nào và ở đâu?

Cấu hình phần cứng

Bảng tính

Lập kế hoạch vận chuyển

Lập kế hoạch nhà máy

quy hoạch sản

Lưu ý rằng nhiều vấn đề trong thế giới thực liên quan đến các biến có giá trị thực

Công thức tìm kiếm tiêu chuẩn (tang dần)

Hãy bắt đầu với cách tiếp cận đơn giản, ngu ngốc, sau đó sửa nó

Các trạng thái được xác định bởi các giá trị được gán cho đến nay

◇ **Trạng thái ban đầu:** nhiệm vụ trống, $\{\}$

◇ **Hàm kế tiếp:** gán giá trị cho biến chưa được gán

điều đó không xung đột với nhiệm vụ hiện tại.

⇒ thất bại nếu không có nhiệm vụ pháp lý nào (không thể sửa được!)

◇ **Kiểm tra mục tiêu:** bài tập hiện tại đã hoàn thành

1) Điều này giống nhau đối với tất cả các CSP! 😊

2) Mọi giải pháp đều xuất hiện ở độ sâu n với các biến n

$\Rightarrow$ sử dụng tìm kiếm theo chiều sâu

3) Đường dẫn không liên quan nên cũng có thể sử dụng công thức trạng thái hoàn chỉnh

4) $b = (n - \ell)d$ ở độ sâu ℓ , do đó $n!d^n$ rồi đi!!!! 😞

Tìm kiếm quay lại

Phép gán biến là *giao hoán*, tức là,

$[WA = red \text{ rồi } NT = green]$ giống như $[NT = green \text{ rồi } WA = red]$

Chỉ cần xem xét việc gán cho một biến duy nhất tại mỗi nút

$\Rightarrow b = d$ và có d^n lá

Tìm kiếm theo chiều sâu cho CSP với các bài tập một biến được gọi là tìm kiếm *quay lại*

Tìm kiếm quay lui là thuật toán cơ bản chưa được hiểu rõ cho CSP

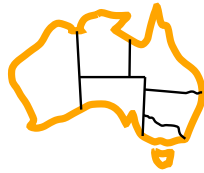
Có thể giải n -quân hậu cho $n \approx 25$

Tìm kiếm quay lại

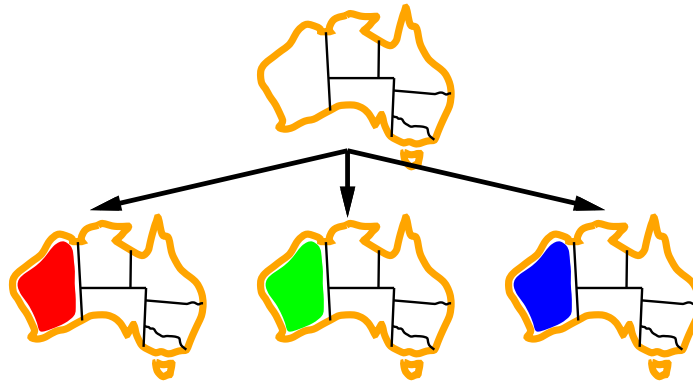
```
function BACKTRACKING-SEARCH(csp) returns solution/failure
return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
if assignment is complete then return assignment
var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
  if value is consistent with assignment given CONSTRAINTS[csp] then
    add {var = value} to assignment
    result ← RECURSIVE-BACKTRACKING(assignment, csp)
    if result ≠ failure then return result
  remove {var = value} from assignment
return failure
```

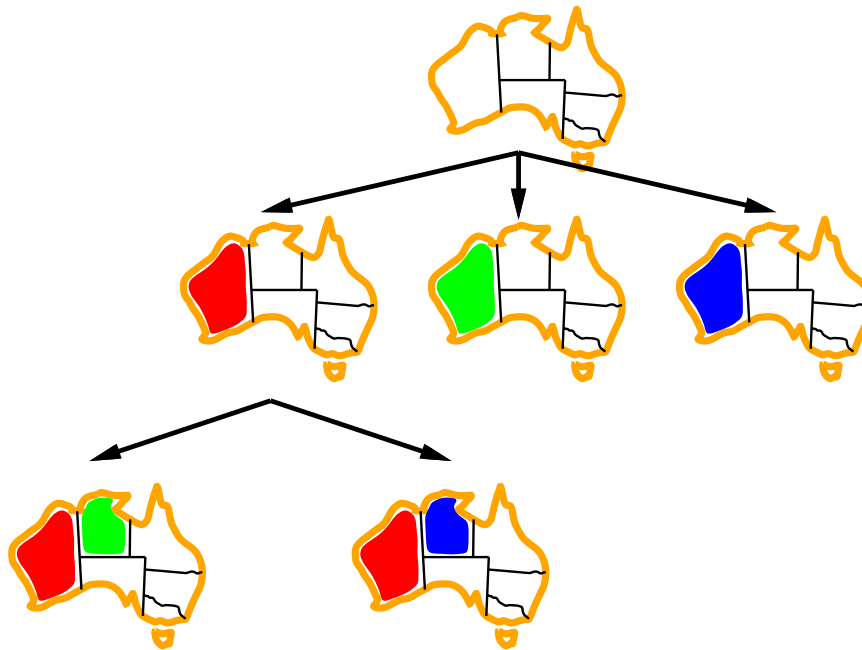
Ví dụ quay lui



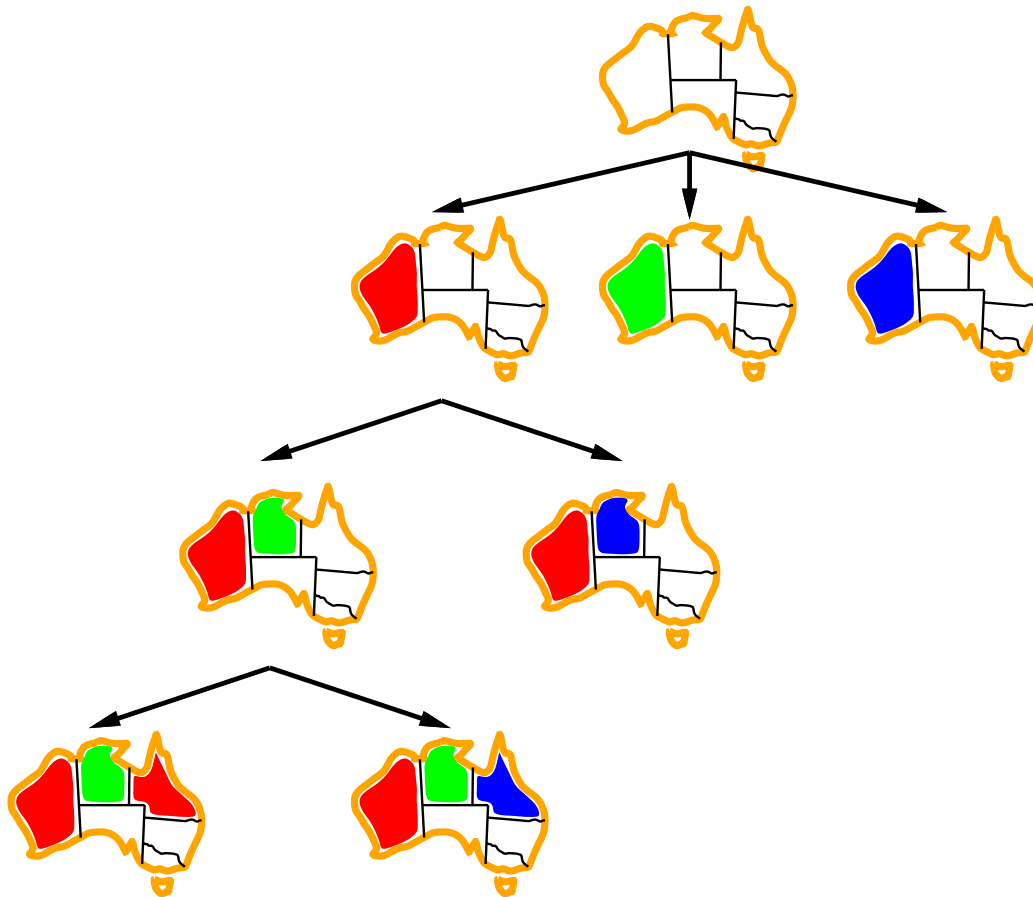
Ví dụ quay lui



Ví dụ quay lui



Ví dụ quay lui



Cải thiện hiệu quả quay lui

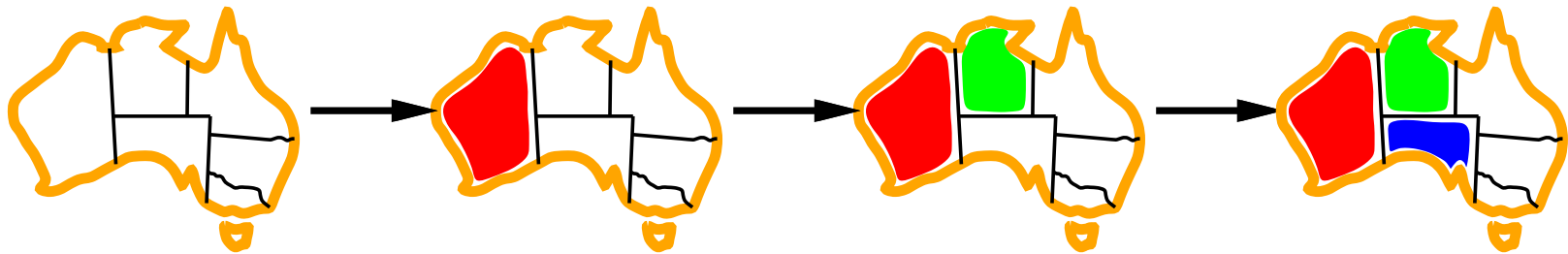
Các phương pháp **Mục đích chung** có thể mang lại tốc độ rất lớn:

1. Biến nào sẽ được gán tiếp theo?
2. Nên thử các giá trị của nó theo thứ tự nào?
3. Liệu chúng ta có thể phát hiện sớm những thất bại không thể tránh khỏi được không?
4. Chúng ta có thể tận dụng được cấu trúc bài toán không?

Giá trị còn lại tối thiểu

Giá trị còn lại tối thiểu (MRV):

chọn biến có giá trị pháp lý nhỏ nhất

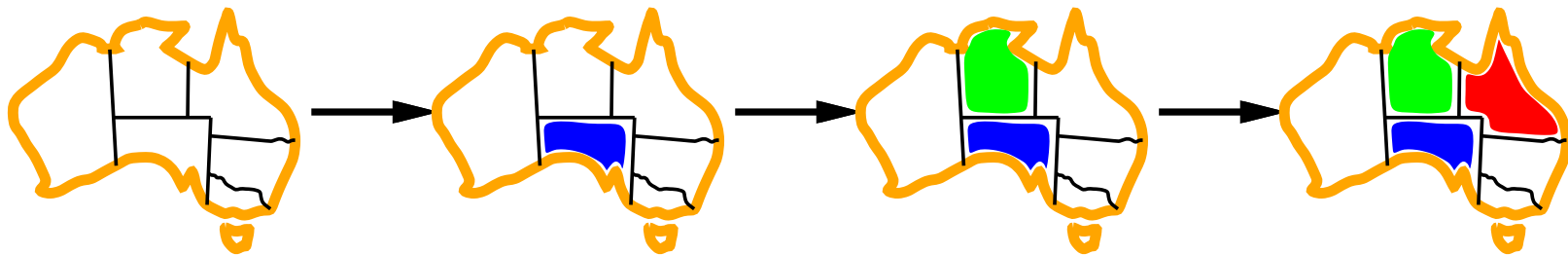


Bậc heuristic

Sự ràng buộc giữa các biến MRV

Heuristic độ:

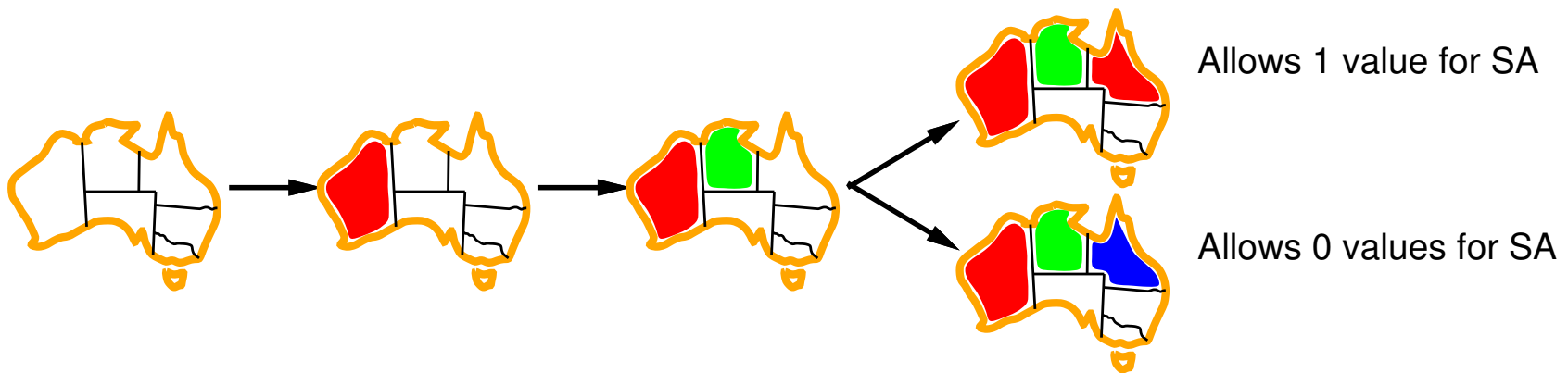
chọn biến có nhiều ràng buộc nhất đối với các biến còn lại



Giá trị ràng buộc nhỏ nhất

Cho một biến, chọn giá trị ít ràng buộc nhất:

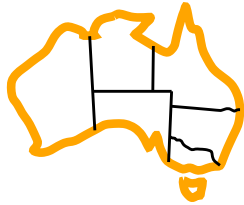
cái loại trừ ít giá trị nhất trong các biến còn lại



Việc kết hợp các phương pháp phỏng đoán này làm cho 1000 quân hậu trở nên khả thi

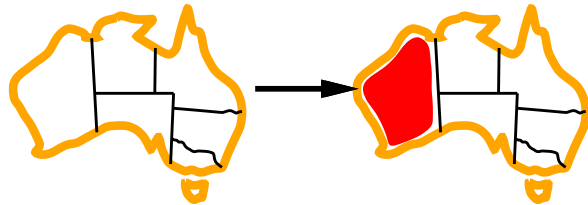
Kiểm tra chuyển tiếp

Idea: Theo dõi các giá trị pháp lý còn lại cho các biến chưa được gán
Chấm dứt tìm kiếm khi bất kỳ biến nào không có giá trị pháp lý



Kiểm tra chuyển tiếp

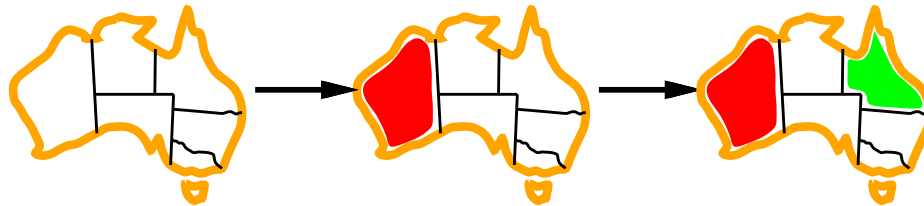
Idea: Theo dõi các giá trị pháp lý còn lại cho các biến chưa được gán
Chấm dứt tìm kiếm khi bất kỳ biến nào không có giá trị pháp lý



WA	NT	Q	NSW	V	SA	T

Kiểm tra chuyển tiếp

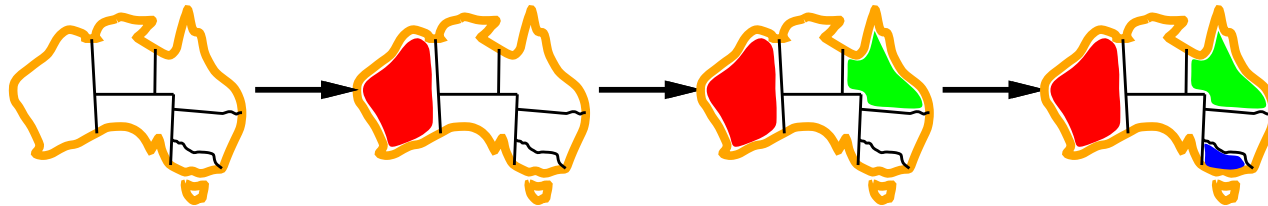
Idea: Theo dõi các giá trị pháp lý còn lại cho các biến chưa được gán
Chấm dứt tìm kiếm khi bất kỳ biến nào không có giá trị pháp lý



WA	NT	Q	NSW	V	SA	T

Kiểm tra chuyển tiếp

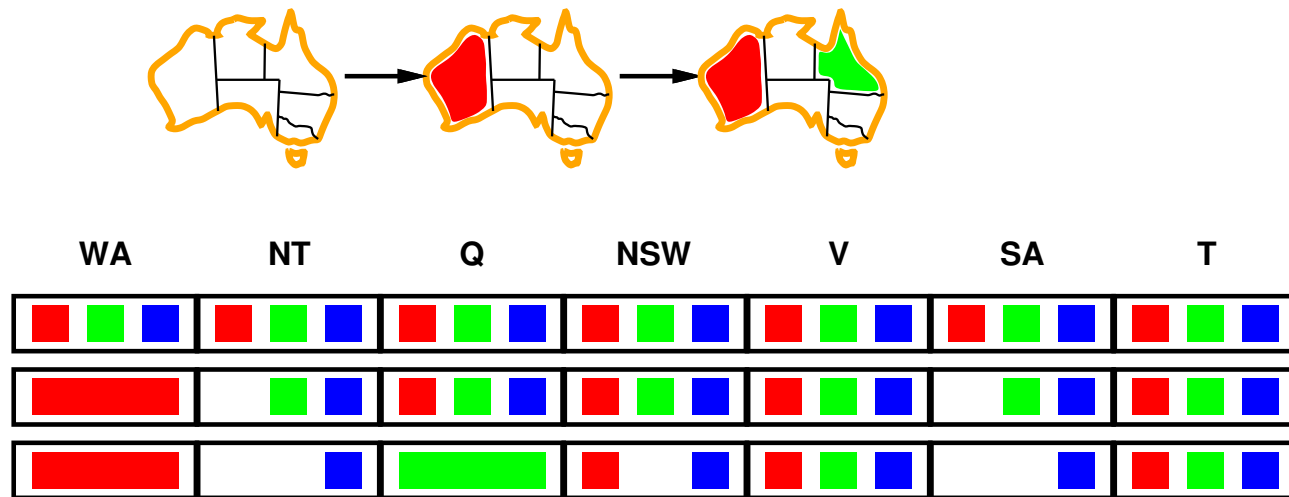
Idea: Theo dõi các giá trị pháp lý còn lại cho các biến chưa được gán
Chấm dứt tìm kiếm khi bất kỳ biến nào không có giá trị pháp lý



WA	NT	Q	NSW	V	SA	T

Truyền bá ràng buộc

Kiểm tra chuyển tiếp truyền thông tin từ các biến được gán đến các biến chưa được gán, nhưng không cung cấp khả năng phát hiện sớm mọi lỗi:



NT và *SA* không thể cùng có màu xanh lam!

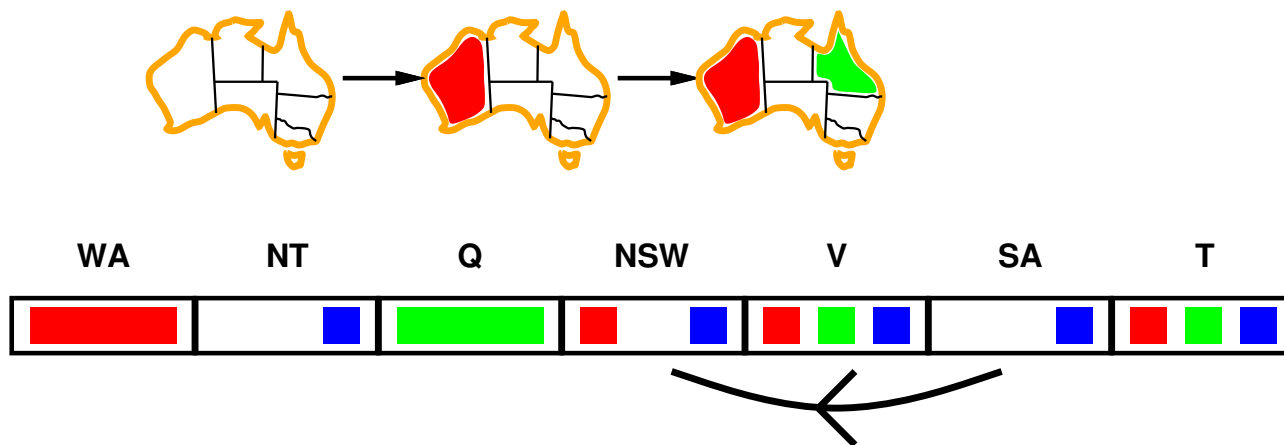
Truyền bá ràng buộc liên tục thực thi các ràng buộc cục bộ

Tính nhất quán của hồ quang

Dạng truyền lan đơn giản nhất làm cho mỗi cung **nhất quán**

$X \rightarrow Y$ nhất quán iff

đối với **mọi** giá trị x của X thì có **some** được phép y

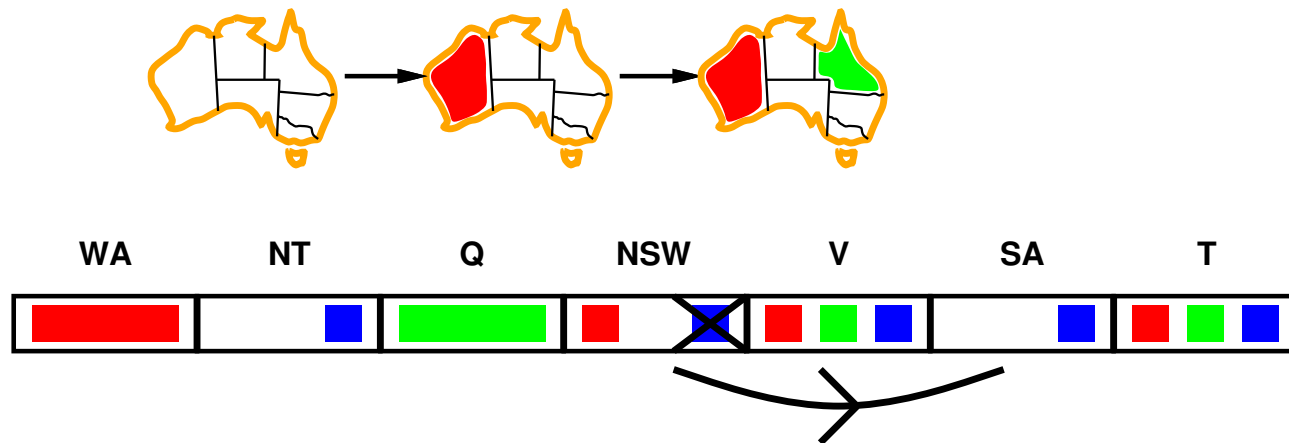


Tính nhất quán của hồ quang

Dạng truyền lan đơn giản nhất làm cho mỗi cung **nhất quán**

$X \rightarrow Y$ nhất quán iff

đối với **mọi** giá trị x của X thì có **some** được phép y

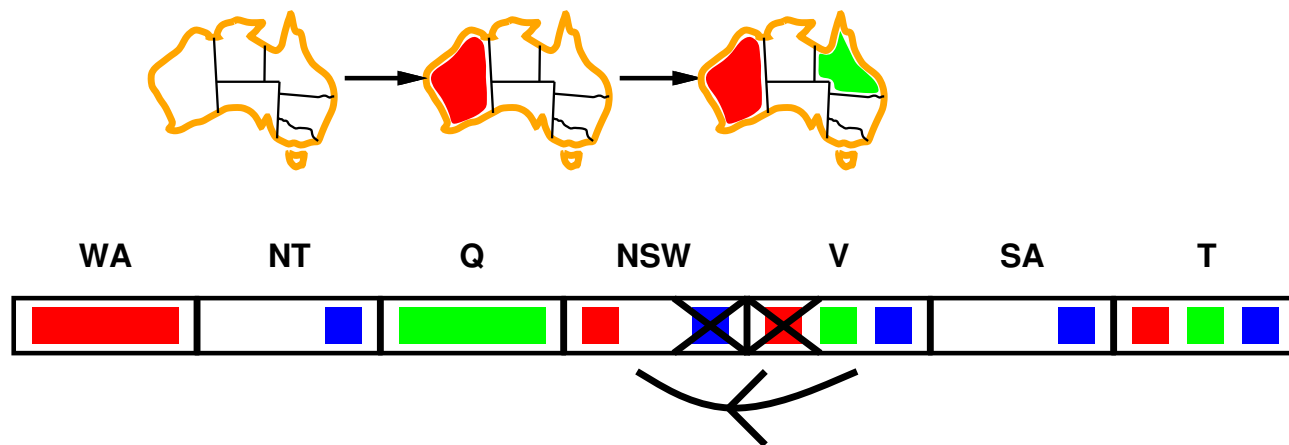


Tính nhất quán của hồ quang

Dạng truyền lan đơn giản nhất làm cho mỗi cung **nhất quán**

$X \rightarrow Y$ nhất quán iff

đối với **mọi** giá trị x của X thì có **some** được phép y



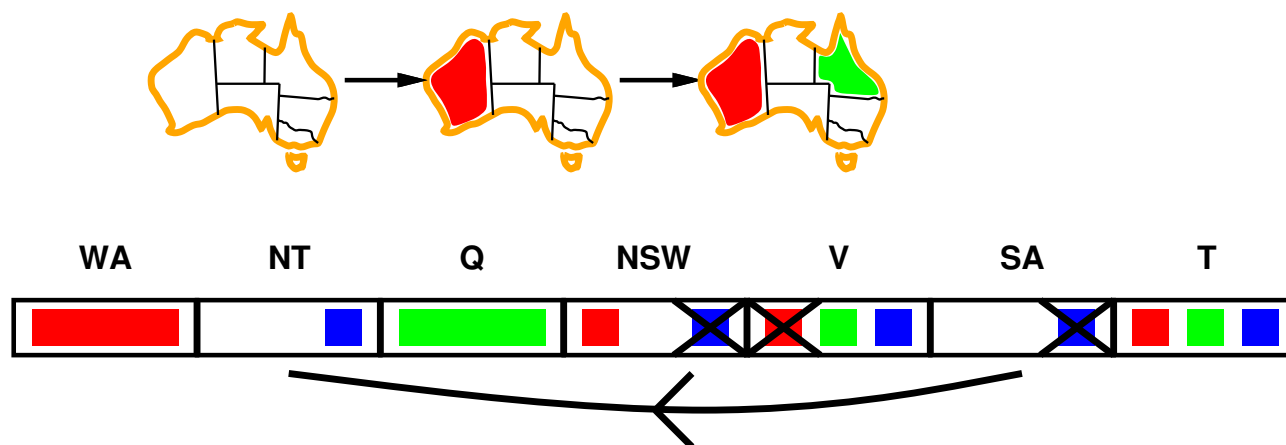
Nếu X mất giá trị, các hàng xóm của X cần được kiểm tra lại

Tính nhất quán của hồ quang

Dạng truyền lan đơn giản nhất làm cho mỗi cung **nhất quán**

$X \rightarrow Y$ nhất quán iff

đối với **mọi** giá trị x của X thì có **some** được phép y



Nếu X mất giá trị, các hàng xóm của X cần được kiểm tra lại

Tính nhất quán của hồ quang phát hiện lỗi sớm hơn việc kiểm tra chuyển tiếp

Có thể chạy như một bộ tiền xử lý hoặc sau mỗi lần gán

Thuật toán nhất quán Arc

function AC-3(*csp*) **returns** the CSP, possibly with reduced domains

inputs: *csp*, a binary CSP with variables $\{X_1, X_2, \dots, X_n\}$

local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do**

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\textit{queue})$

if REMOVE-INCONSISTENT-VALUES(X_i, X_j) **then**

for each X_k **in** NEIGHBORS[X_i] **do**

add (X_k, X_i) to *queue*

function REMOVE-INCONSISTENT-VALUES(X_i, X_j) **returns** true iff succeeds

removed $\leftarrow$ false

for each x **in** DOMAIN[X_i] **do**

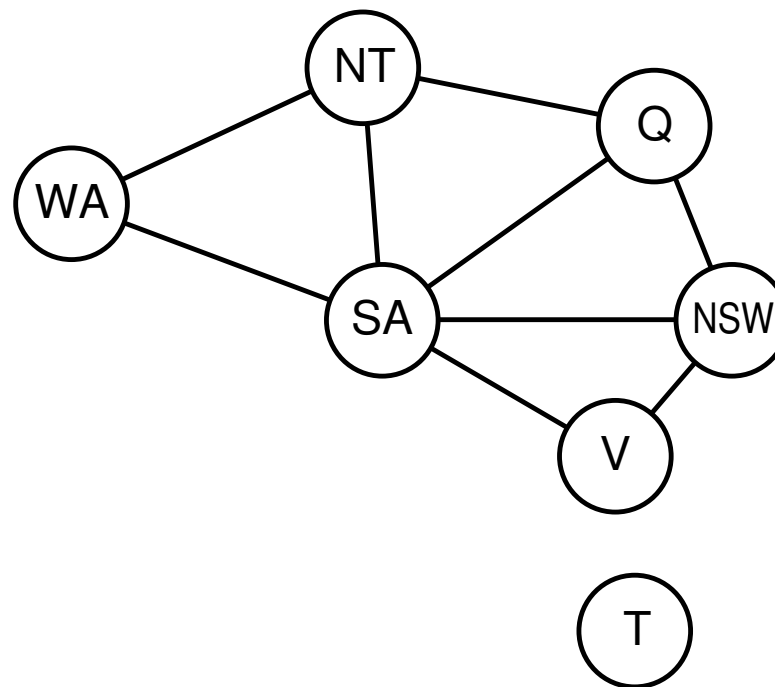
if no value y in DOMAIN[X_j] allows (x, y) to satisfy the constraint $X_i \leftrightarrow X_j$

then delete x from DOMAIN[X_i]; *removed* $\leftarrow$ true

return *removed*

$O(n^2d^3)$, có thể giảm xuống $O(n^2d^2)$ (nhưng việc phát hiện **all** là NP-hard)

Cấu trúc bài toán



Tasmania và đất liền là bài toán con độc lập

Có thể xác định là các thành phần được kết nối của biểu đồ ràng buộc

Cấu trúc bài toán tiếp theo.

Giả sử mỗi bài toán con có các biến c trên tổng số n

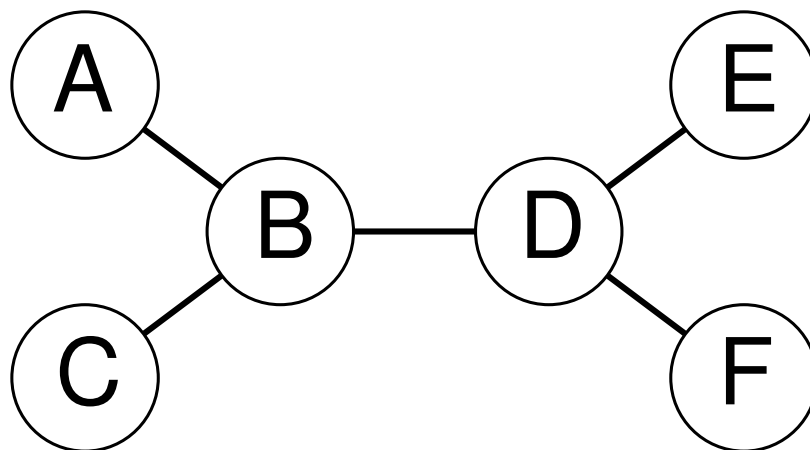
Chi phí giải pháp trong trường hợp xấu nhất là $n/c \cdot d^c$, **tuyến tính** trong n

Ví dụ: $n = 80$, $d = 2$, $c = 20$

$2^{80} = 4$ tỷ năm với tốc độ 10 triệu nút/giây

$4 \cdot 2^{20} = 0,4$ giây ở 10 triệu nút/giây

CSP có cấu trúc cây



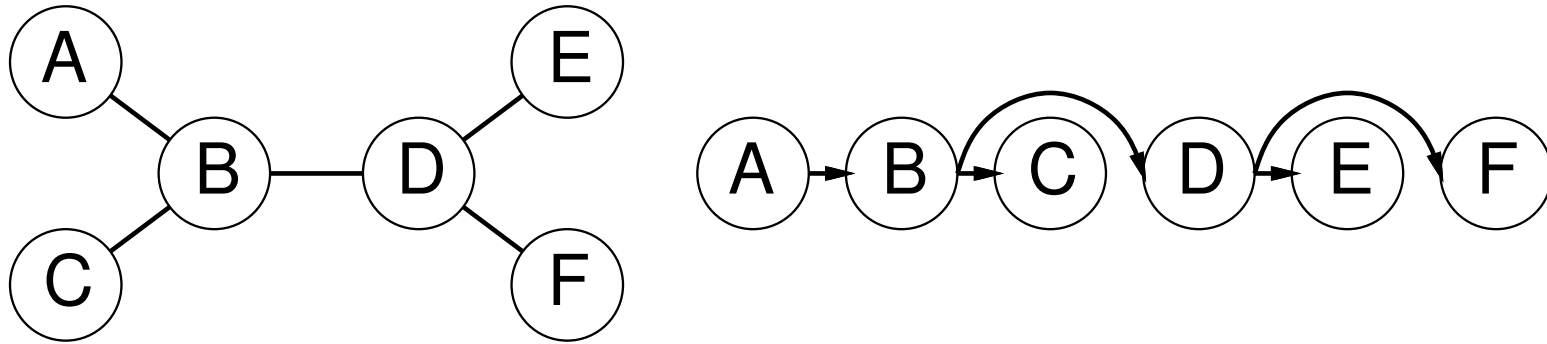
Định lý: nếu đồ thị ràng buộc không có vòng lặp, thì CSP có thể được giải theo $O(nd^2)$ thời gian

So sánh với các CSP chung, trong đó thời gian trong trường hợp xấu nhất là $O(d^n)$

Thuộc tính này cũng áp dụng cho lý luận logic và xác suất: một ví dụ quan trọng về mối quan hệ giữa các hạn chế cú pháp và sự phức tạp của lý luận.

Thuật toán cho CSP có cấu trúc cây

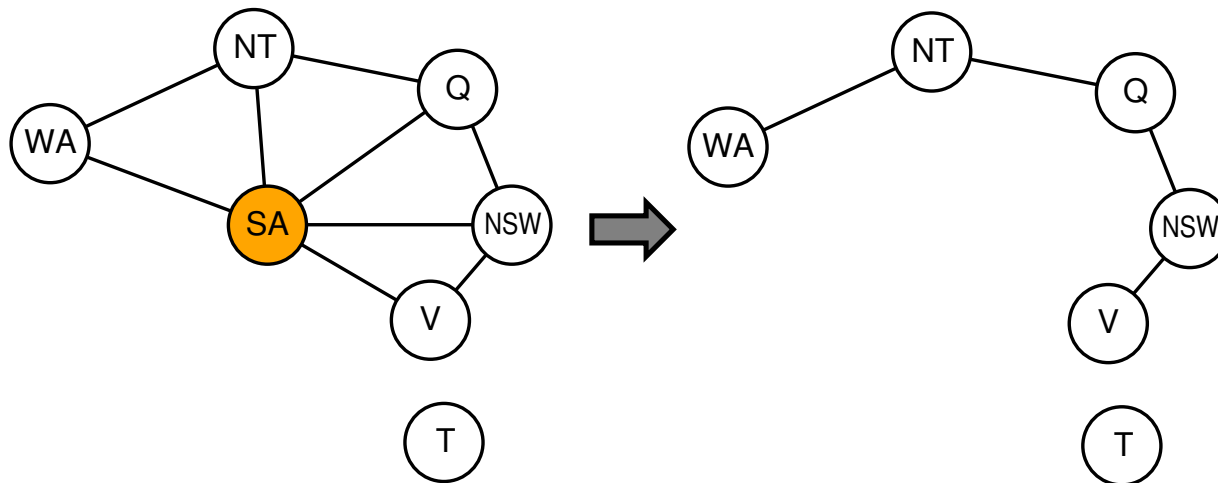
1. Chọn biến làm gốc, sắp xếp biến từ gốc đến lá sao cho mọi nút cha của nút đều đứng trước nó theo thứ tự



2. Đối với j từ n xuống 2 , hãy áp dụng $\text{REMOVEINCONSISTENT}(Parent(X_j), X_j)$
3. Đối với j từ 1 đến n , gán X_j nhất quán với $Parent(X_j)$

CSP gần như có cấu trúc cây

Điều hòa: khởi tạo một biến, cắt tĩa các miền của hàng xóm của nó



Điều hòa cutset: khởi tạo (theo mọi cách) một tập hợp các biến sao cho đồ thị ràng buộc còn lại là một cây

Kích thước cutset $c \Rightarrow$ thời gian chạy $O(d^c \cdot (n - c)d^2)$, rất nhanh đối với c nhỏ

Thuật toán lặp cho CSP

Leo đồi, ủ mô phỏng thường hoạt động với trạng thái “hoàn thành”, tức là, tất cả các biến được gán

Để đăng ký CSP:

cho phép các trạng thái có ràng buộc không thỏa mãn

toán tử **gán lại giá trị biến**

Lựa chọn biến: chọn ngẫu nhiên bất kỳ biến xung đột nào

Lựa chọn giá trị theo **min-conflicts** heuristic:

chọn giá trị vi phạm ít ràng buộc nhất

tức là leo dốc với $h(n) =$ tổng số ràng buộc bị vi phạm

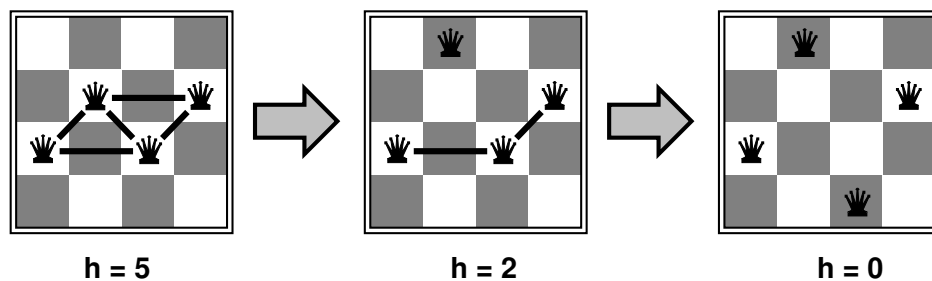
Ví dụ: 4-Queens

Các bang: 4 quân hậu trong 4 cột (các bang $4^4 = 256$)

Toán tử: di chuyển quân hậu trong cột

Kiểm tra mục tiêu: không tấn công

Đánh giá: $h(n) =$ số lần tấn công

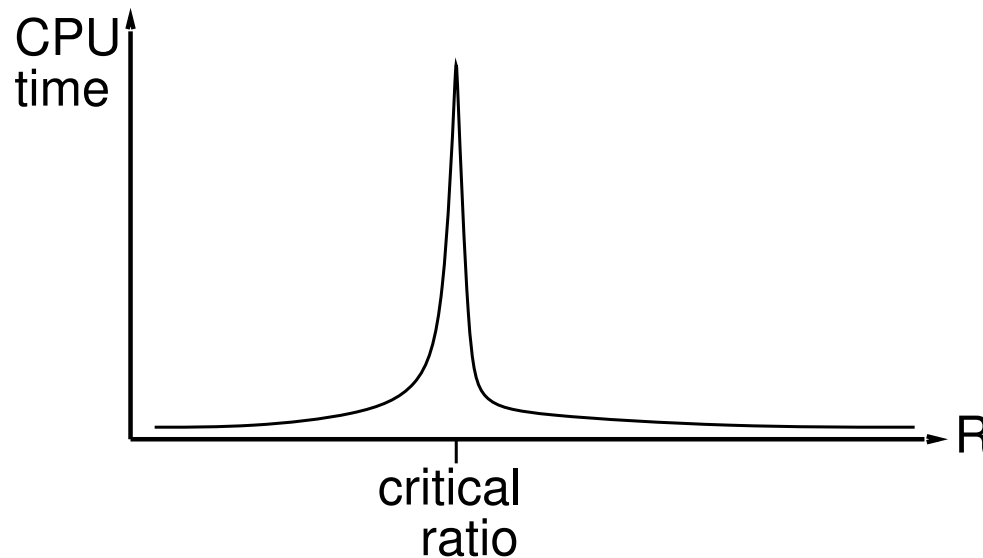


Hiệu suất xung đột tối thiểu

Cho trạng thái ban đầu ngẫu nhiên, có thể giải n -nữ hoàng trong thời gian gần như không đổi cho tùy ý n với xác suất cao (ví dụ: $n = 10.000.000$)

Điều tương tự cũng có vẻ đúng đối với mọi CSP được tạo ngẫu nhiên **ngoại trừ** trong phạm vi tỷ lệ hẹp

$$R = \frac{\text{number of constraints}}{\text{number of variables}}$$



Tóm tắt

CSP là một loại vấn đề đặc biệt:

các trạng thái được xác định bởi các giá trị của một tập hợp biến cố định

kiểm tra mục tiêu được xác định bởi ràng buộc trên các giá trị biến

Quay lui = tìm kiếm theo chiều sâu với một biến được gán cho mỗi nút

Phương pháp phỏng đoán lựa chọn giá trị và thứ tự biến giúp ích đáng kể

Kiểm tra chuyển tiếp ngăn chặn các nhiệm vụ đảm bảo thất bại sau này

Việc truyền bá ràng buộc (ví dụ: tính nhất quán của cung) thực hiện công việc bổ sung

để hạn chế các giá trị và phát hiện sự không nhất quán

Biểu diễn CSP cho phép phân tích cấu trúc vấn đề

CSP có cấu trúc cây có thể được giải quyết trong thời gian tuyến tính

Những xung đột nhỏ lặp đi lặp lại thường có hiệu quả trong thực tế